

Prd Spec

Opportunity Statement

HealthSource intake users frequently upload, print, or scan **pull lists** that must route directly to the Pull List processing queue.

Currently, STORK classification does not always recognize these documents accurately, leading to misroutes, exceptions, and manual rework.

This is not because STORK is expected to be perfect, but because **the system needs better upstream information from the user**. Today, the user has no explicit way to tell HealthSource that a document is a Pull List.

By introducing a “This is a Pull List” option, users can supply clear intent at intake time. This provides STORK and routing services the right context signal, ensuring that automation works in partnership with user input instead of in isolation.

Goal

Enable users to explicitly mark intake documents as “Pull Lists” within existing print, scan, and upload screens.

When selected, the system will bypass STORK classification and route the document directly to the Pull List workflow.

Success means fewer misroutes, faster routing, and higher processor efficiency—while also improving the accuracy and quality of future STORK training data by correctly labeling Pull List documents.

Impact

Business Impact

- Reduce rework caused by misclassified Pull Lists by more than 50%.
- Improve intake-to-correct-queue routing speed by approximately 30%.
- Decrease turnaround time for Pull List processing by removing manual intervention.
- Enhance trust in automation by aligning user input and system logic.

Operational Impact

- Reduce queue backlog from misrouted documents.
- Increase adherence to SLA timing for Pull List routing.
- Provide visibility into Pull List volume, accuracy, and processor behavior via analytics.

Hypotheses

1. **Accuracy Improvement Hypothesis:** If processors can explicitly mark a document as a Pull List, misclassification and rerouting rates will drop by **≥50%** compared to baseline STORK-only routing.
2. **Operational Efficiency Hypothesis:** If Pull List intake bypasses classification, the total routing time from intake to correct queue will improve by **≥30%**.
3. **Human-in-the-Loop Hypothesis:** By allowing the user to declare a document's intent, we can give STORK clearer training signals and reduce false positives in automated classification over time.

Press Release (Future Vision)

HealthSource Introduces Explicit Pull List Intake Option

Today, HealthSource added a simple “This is a Pull List” option to its print, scan, and upload workflows. Processors can now directly identify pull lists, allowing the system to skip classification and immediately route to the correct workflow.

This enhancement doesn't replace automation—it strengthens it. By combining user intent with system intelligence, HealthSource ensures faster routing, less rework, and higher data quality for ongoing STORK improvements.

Operations teams benefit from lower turnaround times and complete audit visibility, while compliance teams gain assurance that all pull list handling is transparent and traceable.

Pre-Mortem: How This Could Go Sideways

- **Incorrect use of the flag:** Users mark non-pull documents as Pull Lists.
Mitigation: Default unchecked; clear helper text; initial training and audit reports.
- **Routing or bypass misconfiguration:** Pull Lists do not land in the correct queue.
Mitigation: Feature toggles and staged rollout with validation checks per site.
- **Low adoption:** UI placement unclear or users unaware of purpose.
Mitigation: Early Alpha testing and iterative UX refinement based on feedback.
- **Incomplete metric instrumentation:** Success difficult to measure.
Mitigation: Event logging validation in pre-production and operational dashboards reviewed before GA.

Prioritized Scope

In Scope

- Add “This is a Pull List” toggle or checkbox to:
 - Print-to-HealthSource

- Scan intake
 - File upload
- When selected:
 - Bypass STORK classification.
 - Auto-assign document type “Pull List” and route to correct queue.
 - Log metadata: user, timestamp, intake type.
 - Include data in reporting and analytics.

Out of Scope

- Retraining STORK classifier for Pull Lists.
- Broader UI redesigns outside existing intake screens.
- Mobile parity (unless already supported).

Acceptance Criteria

- Users can explicitly mark a document as a Pull List across all intake modes.
- When flagged, document bypasses STORK classification and routes directly to Pull List queue.
- All actions are logged for audit and analytics.
- Dashboards display selection rates, exception rates, and turnaround improvements.
- Graceful fallback if routing or STORK services are unavailable.

Dependencies

- HealthSource Intake (UI, ingestion, logging).
- STORK (conditional bypass logic, labeling sync).
- Routing Service (direct mapping to Pull List queue).
- Analytics (data schema updates, dashboards).

Risks & Mitigations

- **False positives:** Operator training; audit validation.
- **Routing failure:** Controlled rollout; automated tests.
- **Compliance gap:** Explicit acceptance test for audit fields.
- **Metric drift:** Ongoing data review during rollout.

Data & Analytics

- Capture:
 - `pull_list_flag_selected`
 - Source: print / scan / upload
 - Timestamps for intake and queue routing
 - Exception/rework events

- Metrics:
 - Pull List accuracy vs. STORK-only baseline
 - Rework volume reduction
 - SLA adherence improvement
- Dashboards:
 - Comparison of flagged vs. unflagged routing success
 - Adoption trends and first-pass accuracy rates

Rollout Strategy

- **Alpha:**
 - Enable for 1–2 high-volume sites.
 - Validate routing accuracy, audit logging, and UI clarity.
- **General Availability (Phased Rollout):**
 - Enable per customer or region with feature toggle.
 - Default on once stability and metrics meet thresholds.

Tech Spec

Purpose

The technical spec / architectural design document helps align our teams on how we will achieve the Product Spec. There are many ways to solve a problem, and this document describes how we'll balance many competing priorities to achieve a solution that delivers optimal business value.

Section Owner:

The engineer(s) assigned to the discovery and scoping of this feature are responsible for owning the content in this section. We invite others to provide input, feedback, and questions on this section.

Order of Operations

The product and engineering teams collaborate on this document to provide increasing levels of clarity around the requirements and best approach that provides the most business value for each situation.

This process will take several distinct tickets and meetings. The goal is to provide initial visibility to the business about risks, opportunities, and costs. As we hone in on identifying exactly what we build and how much time we have available, the later stages involve finalizing the actual technical design that will be built.

While the whole document is needed for a complete review, a "first pass" sizing should always include something simple (even if just a sentence) in the following:

- Design Goals
- One or more proposed architectural designs
- Size Estimate
- Confidence
- Risks/Mitigations table
- Rollout mechanics

Subsequent passes should increase the detail level (accuracy and precision) of the above sections.

During the final pass discovery stage and prior to the kickoff meeting, all remaining sections should be completed as best as possible.

Note: the kickoff meeting must always include Tyler Beach to help satisfy requirements for CASTLE

(<https://datavant.atlassian.net/wiki/spaces/PS/pages/1508179991/CASTLE+for+Engineers>)

System Context

Why: A simple explanation that lists the system components/domains involved and their current state so that readers can be oriented for the remaining context in the document.

What: 1 or 2 paragraphs describing the state of the system as it pertains to the requested change. Links to any previous architecture diagrams or other artifacts helpful in understanding this area.

Design Goals

Why: In order to help align our teams on business value and tradeoffs, we want to explicitly state our design goals that were prioritized when developing the proposed designs.

What: A paragraph or some bullet points of how we balanced such competing forces as, for example:

- Do we build it ourselves, partner with someone, or buy a solution?
- How do we balance quality, cost, and time so that we achieve an optimal business outcome?
- Is a human powered/manual approach better or does automation yield a better balance for the business (short term vs. long term)?
- Is phasing necessary to minimize risk, and what is the appropriate complexity to try to wrangle in the proposed design?
- How important is developer experience/velocity vs. reusability and maintainability of a given change? What is best for the business right now and in the future?
- We are lacking good test coverage in many places in HealthSource. How risky is the change, and how much risk mitigation is necessary as an implicit part of designing the solution?

Proposed Architectural Designs

Why: To help us align our teams on what the change would actually look like, how simple/complex it is, and what pieces of the system are in play for a given change. This is what we're sizing.

What: A collection of text, system diagrams, states, or flows to show what the system would do if we approached the problem a certain way. The description should be in enough detail that the size and complexity of the change can be ascertained. In earlier phases of our design lifecycle, note the options or tradeoffs in each area so that we can collectively align on competing design goals. In later stages of the design, we do not desire ambiguity in this section and want only a single design solution to recommend.

Size Estimate

Why: In order to figure out the right mix of changes for a quarter and also to figure out who are the right people to make the changes, we need to understand an estimate of total cost.

What: Preferably, an estimated number of engineers for an estimated number of total sprints. If we need specific people or teams involved, note that in the cost since dependencies are important contributors to overall cost. Alternatively, hours or t-shirt sizes are acceptable if this is the most accurate we can be. High accuracy is expensive, but we don't want to be irresponsible by being too imprecise.

Confidence

Why: Every project has a unique risk profile and unique tradeoffs/challenges in order to deliver ideal business outcomes, and so we want to try to capture a confidence measure.

What: One or two sentences describing how confident we are about our size estimate.

Risks/Mitigations

Why: Even with the best design, some risks are always present. Call out any open questions or concerns as well as any risks we are well aware of and how we'll defend against them.

What: A table with a list of risks and what we're going to do to reduce the risk (or how the existing design proposed defends against the risk).

Risk	Mitigation

Rollout Mechanics

Why: Delivering business value can rarely/wisely be accomplished in a single step. We want to understand the complexity and steps needed to deliver value to customers, what is delivered in each phase, and how what might need to be communicated at each step of the process.

What: A series of steps, an outline, or a sequence diagram showing how we'd release the changes, what things might need to be migrated/released in each stage, and how we'd rollout the functionality to users. The focus is on what technical changes are needed and what business value users will be delivered during each step.

Note any important system or functional criteria that should be true before advancing to the next phase.

If rollback is for some reason not possible in a step, note that here so we can understand the implications.

This section is intended to be complementary to the **Release Strategy** tab.

Security Considerations and Implications

Why: We want to make sure that all changes take security implications into account and that the risk of those changes is called out.

What: A paragraph or bullet point list describing the known security risks or places where the change interacts with important data, encryption, or auth. We follow the CASTLE process, and this section is not meant to replace that. Tyler Beach is responsible for assessing overall security risk.

Monitoring, Alerting, and Measuring

Why: All software changes need to be supported by the team, and this means having a good monitoring and alerting strategy for the change. Likewise, we want to make sure we're measuring both platform level metrics as well as business level metrics.

What: A bullet point list of what things we'd measure, where we'd measure it (Datadog, Sigma, or a manual query) and alert on. The severity/response plan of the metric should be noted so that we can understand what things wake people up vs. just are used to track adoption.

Testing

Why: All changes must be tested, but not all changes have the same risk level when developing tests. This section helps us align on how deep we should go on which areas of testing the change.

What: A list of specific tests or strategies that we will use to verify the changes, and if applicable, which parts might be done manually vs. in an automated fashion.

Dependencies

Why: Some changes will require new permissions, access, or important work from other teams in order to be complete. We should note that here.

What: A bullet point list of external dependencies that we will need in order for this project to be successful.

